INT161 · Basic Backend Development

Express Framework

Express แทน HTTP code ที่เขียนเอง: routing, middleware และ layer Controller, Service, Repository

Raw http vs Express | Reading the Request | Handling Responses | Controller, Service, Repository | Routing | Middleware | Project Template | อภิธานศัพท์

01

Raw http vs Express

Express เป็น **framework** ขนาดเล็กบน http **module** งานเหมือนเดิม แต่ตัดส่วนที่ต้องเขียนเองออก

Reading the body

http: Collect stream chunks by hand

Express: `express.json()`

Matching the URL

http: if / else on request.url

Express: `app.put('/api/subjects/:id', ...)`

Reading query and path

http: new URL(...) and split

Express: `req.query and req.params`

Sending the answer

http: writeHead + JSON.stringify + end

Express: `res.status().json()`

Last class: reading the body by hand

```
function getBody(request) {
  return new Promise((resolve, reject) => {
    let body = '';
    request.on('data', chunk => { body += chunk.toString(); });
    request.on('end', () => resolve(body ? JSON.parse(body) : null));
    request.on('error', reject);
  });
}
```

app.js: the same job in Express

```
import express from 'express';
const app = express();
app.use(express.json()); // middleware
app.put('/api/subjects/:id', (req, res) => {
  const { filter, page, size } = req.query;
  const id = req.params.id;
  const subject = req.body;
  subject.id = id;
  res.json([subject, { filter, page, size }]);
});
app.listen(3000);
```

- framework คือเครื่องมือที่มีกรอบ ต้องเขียนตามกรอบ จึงต้องเข้าใจว่ามันทำงานอย่างไร
- Express ไม่บังคับ database หรือโครงสร้าง folder
- หัวใจคือ **routing** และ **middleware** ที่เหลือของคลาสต่อยอดจากสองอย่างนี้

> **Postman**
>
> **browser** พิมพ์ที่ address bar ส่งได้แค่ GET ส่วน **Postman** เลือก PUT, POST หรือ DELETE ได้ ใช้ทดสอบ **API** เพราะ **front end** มักยังไม่เสร็จ

02

Reading the Request

request พกข้อมูลมาสามที่ Express แยกให้เป็น property ของใครของมัน

Path | Query | Body

```
PUT /api/subjects/5?page=2&size=10
{ "title": "Web Technology", "credit": 1 }
```

อ่านด้วย

`req.params.id`

ได้ค่า

`"5"`

:id ใน **route** pattern บอกว่าเป็น subject ไหน

03

Handling Responses

res.json() ส่งค่า JavaScript เป็น **JSON** ส่วน res.status() เลือก **status code** ก่อน

Sending a response

```
res.json(subject);                  // 200 by default
res.status(201).json(subject);      // set the code, then send
res.status(404).json({ message: 'Subject not found' });
res.status(404);                    // sets the code only, nothing is sent yet
```

⚠ ระวัง

res.status() อย่างเดียวไม่ได้ส่งอะไร มันคืน res เพื่อให้ต่อ .json() หรือ .send() ได้

04

Controller, Service, Repository

อาจารย์เปรียบ **backend** เป็นโรงพยาบาล request คือคนไข้ เลือก layer ดู

Controller | Service | Repository

Client → Controller → Service → Repository → Database

Controller = โต๊ะ reception

รับ **HTTP request** ส่งต่อ และส่ง HTTP **response**

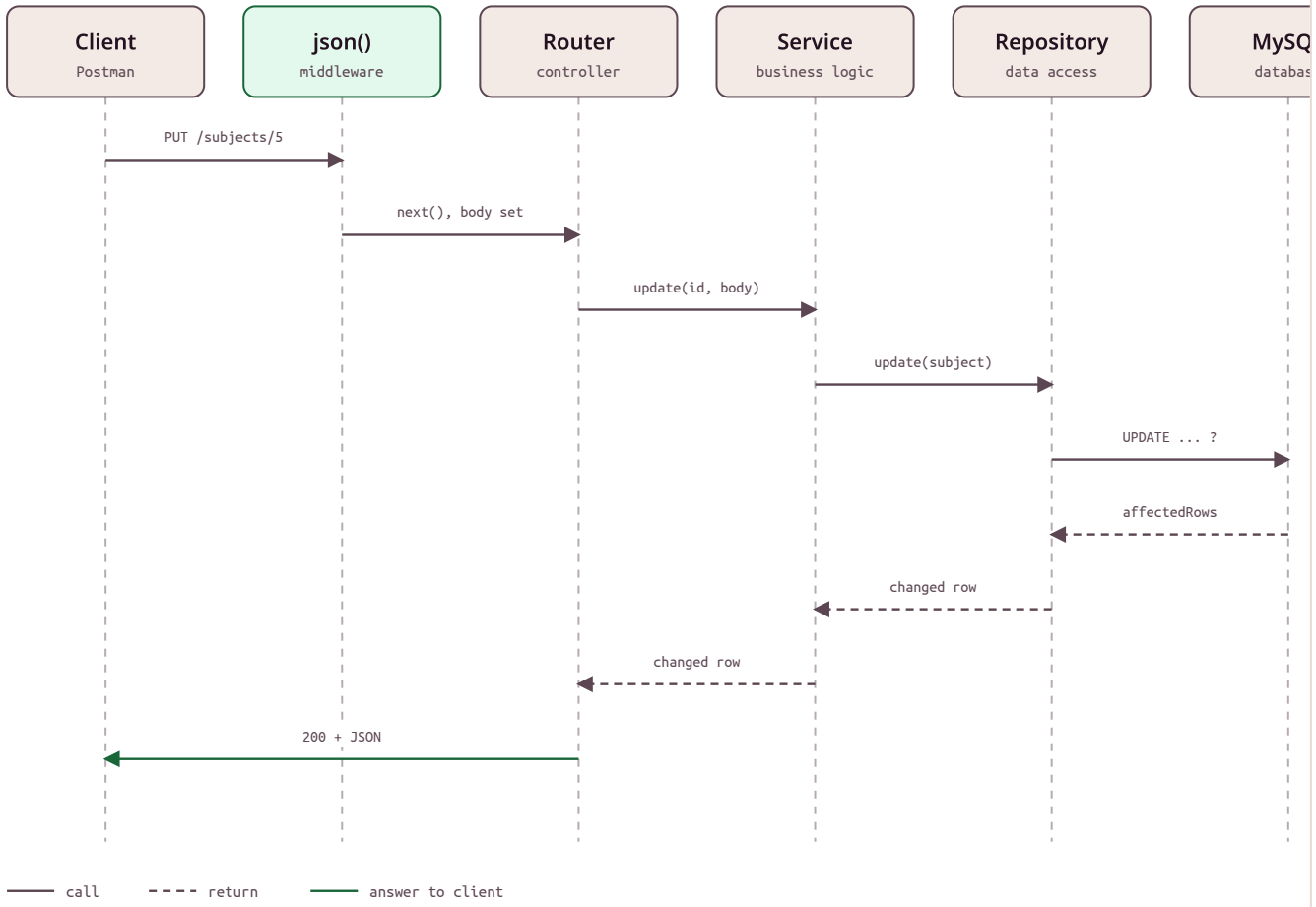
ควรมี

Routing, mapping, validation

ไม่ควรมี

Business logic, DB code

```
res.status(404).json({ message: 'Subject not found' })
```



- ทำไมสามชั้น เพื่อไม่ต้องเขียน code ซ้ำ student **service** เรียก subject service ได้เลย

- แต่ละ layer คุยกับ layer ที่อยู่ถัดลงไปเท่านั้น ขยายและแก้ไขง่าย

05

Routing

route คือ METHOD, **PATH**, HANDLER รวม route ของ **resource** หนึ่งไว้ใน **Router** แล้ว mount ใต้ prefix

routes/subject-route.js

```
import express from 'express';
import * as service from '../services/subject-service.js';
const router = express.Router();
router.get('/', async (req, res, next) => {
  const subjects = await service.getAllSubjects();
  res.json(subjects);
});
router.get('/:id', async (req, res, next) => {
  const { id } = req.params;
  const [subject] = await service.getSubjectById(id);
  if (!subject[0]) return res.status(404).json({ message: 'Subject not found' });
  res.json(subject[0]);
});
export default router;
```

app.js: mounting routers

```
import express from 'express';
const PORT = 3000;
const app = express();
app.use(express.json());
app.use('/customers', customerRouter);
app.use('/orders', orderRouter);
app.use('/products', productRouter);
app.listen(PORT, () => {
  console.log('Running at http://localhost:' + PORT);
});
```

handler อ่าน request แล้วเรียก service ไม่เก็บ **business logic**

06

Middleware

middleware คือฟังก์ชันที่คั่นระหว่าง request กับ router ลำดับสำคัญ ลองสามแบบ

json แล้ว router | router แล้ว json | ไม่มี json

Client → express.json() → Router → Response

ใน route handler

```
req.body = { title: "Web Technology"
}
```

ผลลัพธ์

200 บันทึก subject แล้ว

request ผ่าน **middleware** ตามลำดับที่เขียน **body** ถูกแปลงก่อนที่ **router** จะอ่าน

A logger middleware

```
app.use((req, res, next) => {
  console.log(req.method + " " + req.url);
  next(); // pass it on, or the request hangs
});
```

- เรียก **next()** เพื่อไปต่อ หรือส่ง **response** เพื่อหยุดสาย
- งานทั่วไป: แปลง JSON, ตรวจ login, log, จัดการ error, เปิด CORS

Application-level | Router-level | Error-handling | Built-in | Third-party

07

Project Template

คอร์สนี้เขียน **ESM** ตั้งค่าครั้งเดียวแล้วเซฟโปรเจกต์เป็น template

package.json

```
{
  "name": "basic-rest-express",
  "version": "1.0.0",
  "main": "src/app.js",
  "type": "module"
}
```

1. สร้างโปรเจกต์แล้วรัน **npm** install express
2. เพิ่ม "type": "module" ใน package.json
3. สร้าง src ที่มี configs, repositories, services และ routes
4. เขียน app.js, router, service และ **repository**
5. ใน WebStorm: Tools, Save project as template ตั้งชื่อ Express-ESM

08

อภิธานศัพท์

API
: Application Programming Interface: ช่องทางที่โปรแกรมเปิดให้โปรแกรมอื่นเรียกใช้

backend
: ส่วนบน server จัดการข้อมูล กฎ และ database แล้วตอบ front end ด้วย JSON

body
: ข้อมูลที่ request พกมาหลัง header สำหรับ API มักเป็น JSON

browser
: client ที่คนส่วนใหญ่ใช้ เช่น Chrome, Edge, Safari

business logic
: กฎของแอป เช่น การตรวจสอบ การคำนวณ และสิ่งที่ทำได้เมื่อไร

ESM
: ระบบ module สมัยใหม่ ใช้ import และ export ต้องมี "type": "module" ใน package.json

Express
: web framework ขนาดเล็กของ Node.js เพิ่ม routing และ middleware บน http module ที่มีมาให้

framework
: ชุดเครื่องมือที่ช่วยให้งานง่ายขึ้น แต่มีกรอบที่เราต้องเขียนตาม

front end
: ส่วนที่ผู้ใช้เห็นและใช้ รันบน browser หรือโทรศัพท์

HTTP
: HyperText Transfer Protocol: ภาษาข้อความธรรมดาที่ browser กับ web server ใช้คุยกัน ฝั่งหนึ่งส่ง request อีกฝั่งส่ง response

JSON
: JavaScript Object Notation: รูปแบบข้อมูลที่เป็นข้อความ คนและโปรแกรมอ่านได้

middleware
: ฟังก์ชัน (req, res, next) ที่ทำงานระหว่าง request กับ route handler แก้ request ตอบเอง หรือส่งต่อได้

module
: ไฟล์ code ที่โหลดเข้าไฟล์อื่นด้วย import

next()
: เรียกเพื่อส่ง request ต่อให้ middleware ตัวถัดไป

npm
: Node Package Manager: ติดตั้ง library เช่น express ลงใน node_modules

PATH

ค่าตั้งของ Windows ที่ระบุโฟลเดอร์ซึ่งใช้ค้นหาคำสั่งที่พิมพ์ ถ้า bin ของ MySQL ไม่อยู่ในนี้ จะไม่รู้จักคำสั่ง mysql

Postman

เครื่องมือส่ง HTTP method ใดก็ได้ไปที่ API ใช้ทดสอบก่อนที่ front end จะเสร็จ

repository

layer ที่อ่านเขียน database ไม่มี business rule และไม่มี HTTP

request

สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

resource

สิ่งที่ API ให้เข้าถึง เช่น product หรือ customer ในวิชานี้ตรงกับ table ใน database

response

สิ่งที่ server ส่งกลับ: status code, header และ body

route

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

Router

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

routing

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

service

layer ที่เก็บ business rule เรียก repository และไม่รู้เรื่อง HTTP

status code

ตัวเลขที่บอก client ว่าผลเป็นอย่างไร เช่น 200 OK, 201 Created, 404 Not Found, 500 Server Error